

LABORATORIO 5 – PUBLICACIÓN DE API REST AWS Lambda + API Gateway

Integrantes del Equipo

Antes de iniciar el laboratorio, complete la siguiente información:

1. Roberto Josué Polanco Hernández
2. Roberto Carlos Moran Gómez
3. Christopher Alexander Marroquin Figueroa
4. Oscar José Pleites Lemus
5. Angie Fiorella Alberto Guzman
6. Diego Francisco Arévalo Miranda

Reglas de Laboratorio

- Todo recurso AWS deberá ser creado utilizando Terraform.
- No está permitido crear recursos manualmente desde la consola de AWS.
- La consola únicamente podrá utilizarse para:
 - Verificación de recursos desplegados.
 - Obtención de evidencias.
 - Consulta de logs.
- Cualquier recurso creado manualmente será considerado incorrecto.
- Todos los integrantes deben comprender el funcionamiento completo de la solución.
- El laboratorio será expuesto individualmente.

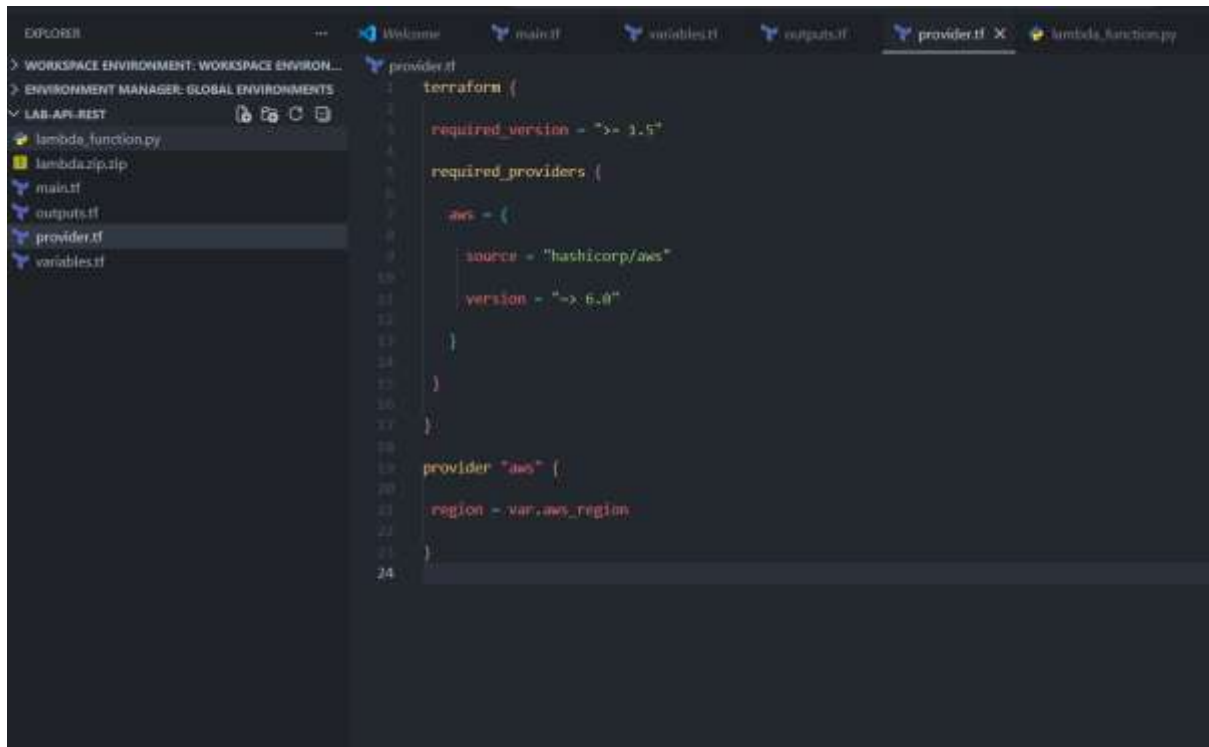
Evidencias del Laboratorio

Evidencia 1

Repositorio o estructura de archivos Terraform mostrando:

- main.tf
- variables.tf
- outputs.tf

- provider.tf
- lambda_function.py



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like lambda_function.py, lambda.zip, main.tf, outputs.tf, provider.tf, and variables.tf. The code editor displays the content of provider.tf, which is a Terraform configuration for the AWS provider.

```
1 terraform {  
2   required_version = ">= 1.5"  
3  
4   required_providers {  
5  
6     aws = {  
7       source = "hashicorp/aws"  
8       version = "~> 6.0"  
9     }  
10  }  
11 }  
12  
13 provider "aws" {  
14   region = var.aws_region  
15 }  
16  
17  
18  
19  
20  
21  
22  
23  
24
```

Evidencia 2

Resultado exitoso de:

terraform validate

mostrando:

Success! The configuration is valid.

The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying a project structure for 'LAB-API-REST'. The main editor window shows the 'main.tf' file with Terraform code for creating an AWS IAM role. The code defines a resource 'aws_iam_role' named 'hello-lambda-role' with an 'assume_role_policy' that allows the 'sts:AssumeRole' action on the 'lambda.amazonaws.com' service. The terminal at the bottom shows the command 'terraform validate' being executed, resulting in a 'Success! the configuration is valid.' message.

```
1 // Creacion de IAM Role
2 resource "aws_iam_role" "lambda_role" {
3
4   name = "hello-lambda-role"
5
6   assume_role_policy = jsonencode({
7
8     version = "2012-10-17"
9
10    statement = [
11
12      {
13
14        effect = "Allow"
15
16        action = "sts:AssumeRole"
17
18        principal = {
19
20          service = "lambda.amazonaws.com"
21        }
22      }
23    ]
24  })
25 }
```

Terminal Output:

```
C:\Users\chris\Documents\Programming\I940\Cloud\lab-api-rest>terraform validate
Success! the configuration is valid.

C:\Users\chris\Documents\Programming\I940\Cloud\lab-api-rest>
```

The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the same project structure. The main editor window shows the 'main.tf' file with Terraform code for creating an AWS Lambda permission. The code defines a resource 'aws_lambda_permission' named 'allow_gateway' with a 'statement_id' of 'AllowExecutionFromAPIGateway', an 'action' of 'lambda:InvokeFunction', and a 'principal' of 'apigateway.amazonaws.com'. The 'source_arn' is set to the execution ARN of the 'hello-api' REST API. The terminal at the bottom shows the command 'terraform validate' being executed, resulting in a 'Success! the configuration is valid.' message.

```
111 // Creacion de IAM Role
112 resource "aws_iam_role" "lambda_role" {
113
114   name = "hello-lambda-role"
115
116   assume_role_policy = jsonencode({
117
118     version = "2012-10-17"
119
120    statement = [
121
122      {
123
124        effect = "Allow"
125
126        action = "sts:AssumeRole"
127
128        principal = {
129
130          service = "lambda.amazonaws.com"
131        }
132      }
133    ]
134  })
135 }
```

Terminal Output:

```
C:\Users\chris\Documents\Programming\I940\Cloud\lab-api-rest>terraform validate
Success! the configuration is valid.

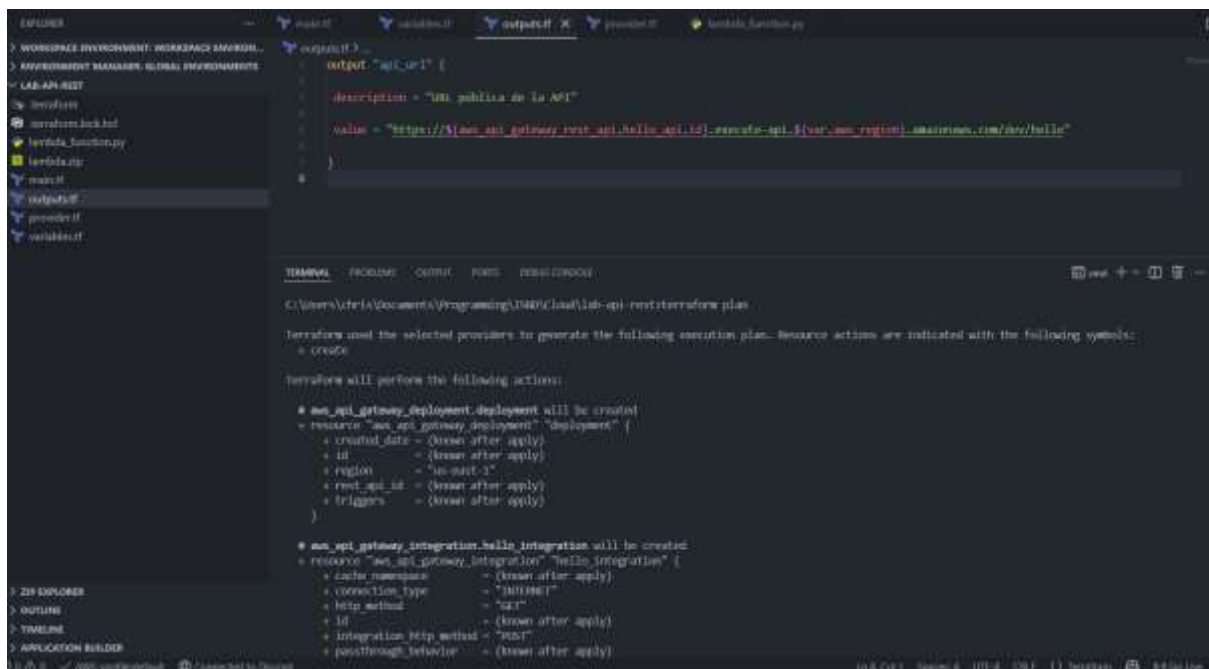
C:\Users\chris\Documents\Programming\I940\Cloud\lab-api-rest>
```

Evidencia 3

Resultado exitoso de:

terraform plan

mostrando los recursos que serán creados.



```
terraform plan

C:\Users\cfe\Documents\Programming\IAD\Cloud\lab-api>terraform plan

terraform used the selected providers to generate the following execution plan... Resource actions are indicated with the following symbols:
+ create

terraform will perform the following actions:

+ aws_api_gateway_deployment.deployment will be created
+ resource "aws_api_gateway_deployment" "deployment" {
  + created_date = (known after apply)
  + id          = (known after apply)
  + region     = "us-east-1"
  + rest_api_id = (known after apply)
  + triggers   = (known after apply)
}

+ aws_api_gateway_integration.hello_integration will be created
+ resource "aws_api_gateway_integration" "hello_integration" {
  + lambda_namespace = (known after apply)
  + connection_type  = "MURDER"
  + http_method      = "GET"
  + id               = (known after apply)
  + integration_http_method = "POST"
  + passthrough_behavior = (known after apply)
}
```

Evidencia 4

Análisis del plan de Terraform identificando:

- Cantidad de recursos a crear.
- Nombre del IAM Role.
- Nombre de la función Lambda.
- Nombre de la API.
- Nombre del Stage.
-

Elemento	Valor
IAM Role	"hello-lambda-role"
Lambda	"hello-lambda"
API	"hello-api"
Stage	"dev"

Recursos a crear

10

The screenshot shows a VS Code editor with a Terraform project. The Explorer sidebar on the left lists files: .terraform, .terraform.lock.hcl, lambda_function.py, lambda.zip, main.tf, outputs.tf, provider.tf, and variables.tf. The main editor displays the contents of outputs.tf, which defines an output named 'api_url' with a description 'URL pública de la API' and a value using a URL template. Below the editor, the TERMINAL panel shows the output of a Terraform command, displaying a plan for creating an 'aws_lambda_permission.allow_gateway' resource. The plan lists attributes like action, function_name, id, principal, region, source_arn, statement_id, and statement_id_prefix. It also shows changes to the 'api_url' output. At the bottom, a note states: 'Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly'.

```
output "api_url" {
  description = "URL pública de la API"
  value = "https://{aws_api_gateway_rest_api.hello_api.id}.execute-api.${var.aws_region}..."
}
```

```
# aws_lambda_permission.allow_gateway will be created
+ resource "aws_lambda_permission" "allow_gateway" {
  + action          = "lambda:InvokeFunction"
  + function_name   = "hello-lambda"
  + id              = (known after apply)
  + principal       = "apigateway.amazonaws.com"
  + region          = "us-east-1"
  + source_arn      = (known after apply)
  + statement_id    = "AllowExecutionFromAPIGateway"
  + statement_id_prefix = (known after apply)
}
```

Plan: 10 to add, 0 to change, 0 to destroy.

Changes to Outputs:

- api_url = (known after apply)

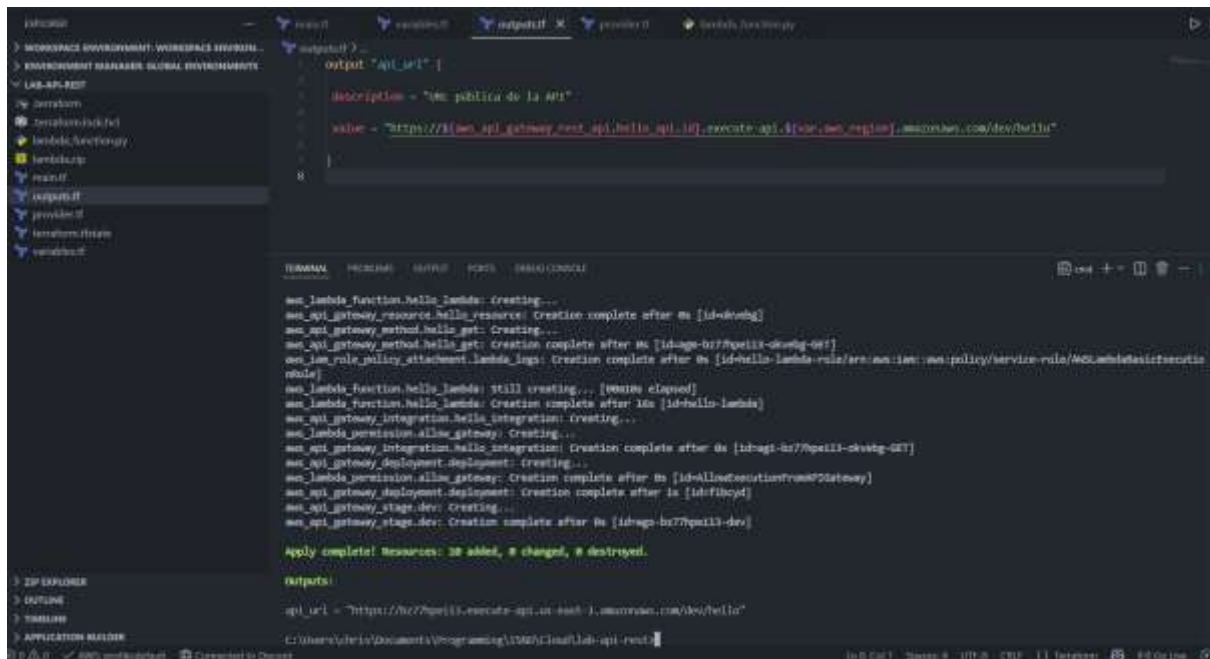
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly

Evidencia 5

Resultado exitoso de:

terraform apply

mostrando la creación de todos los recursos.

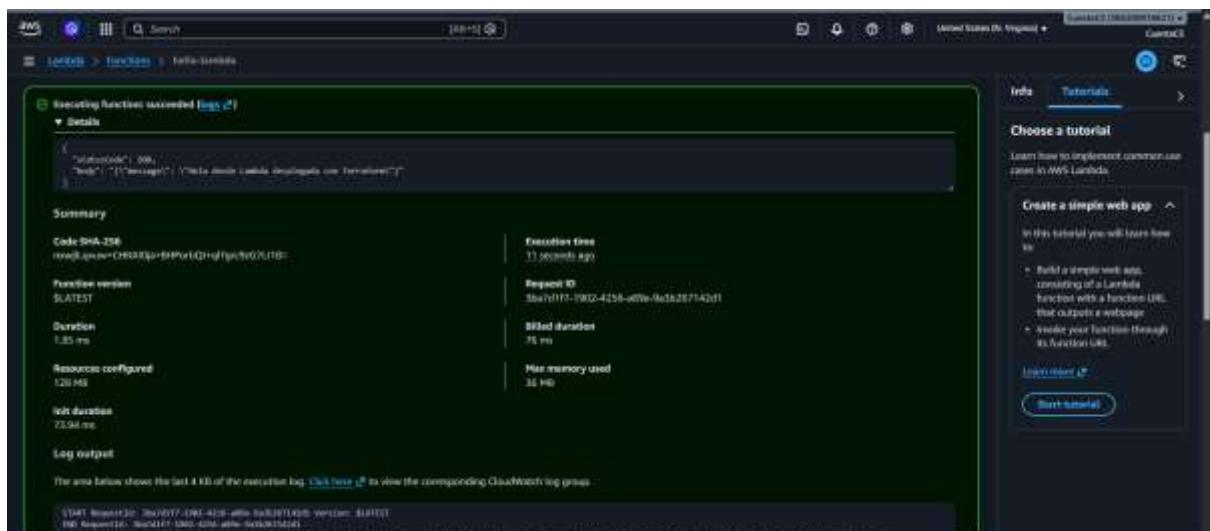


Evidencia 6

Prueba de ejecución directa de la función Lambda desde AWS.

Capturar:

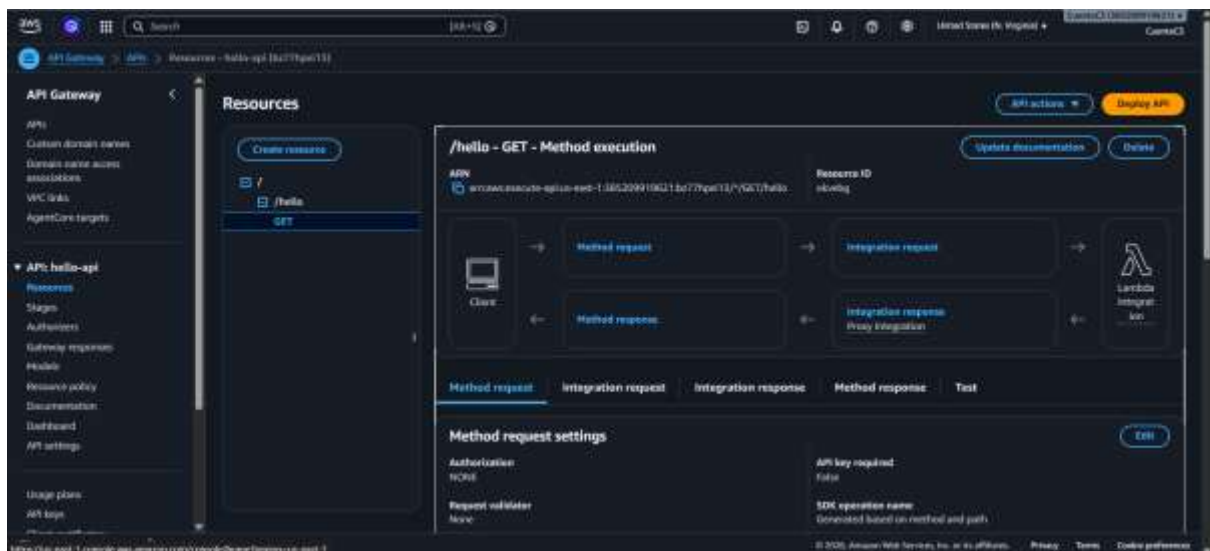
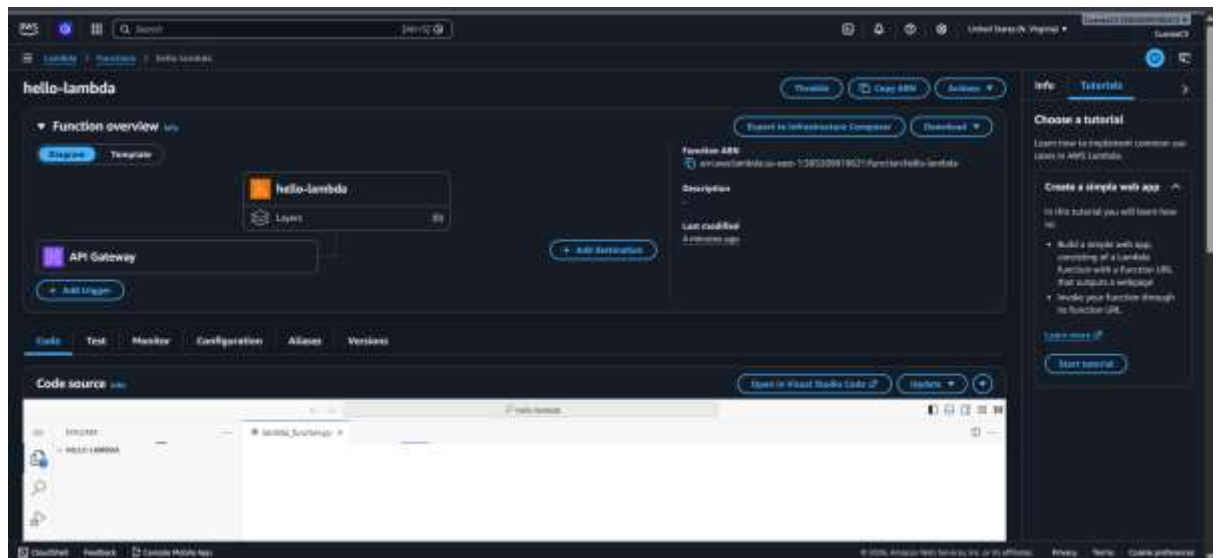
- Resultado de ejecución.
- Duration.
- Billed Duration.

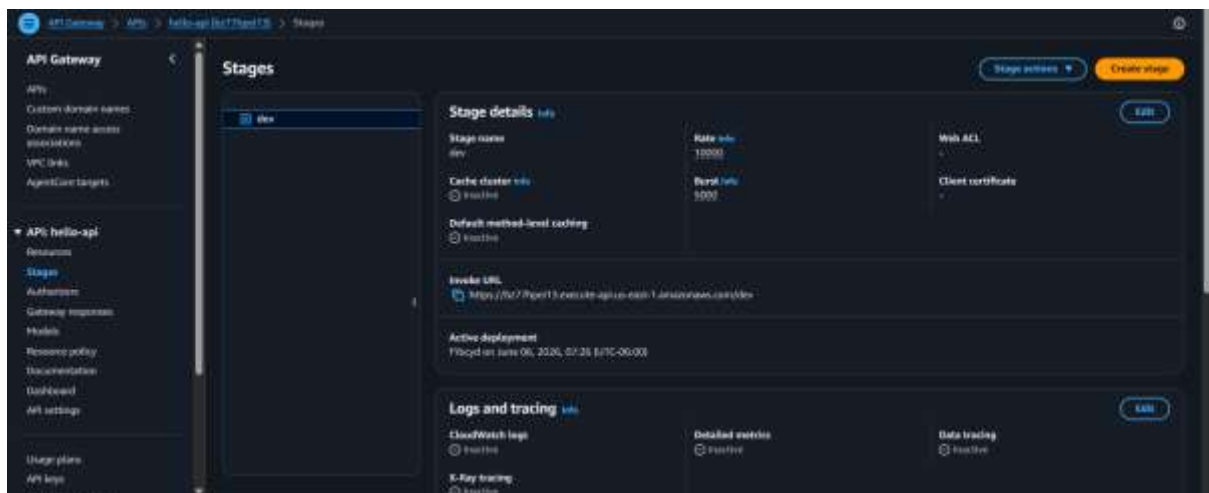
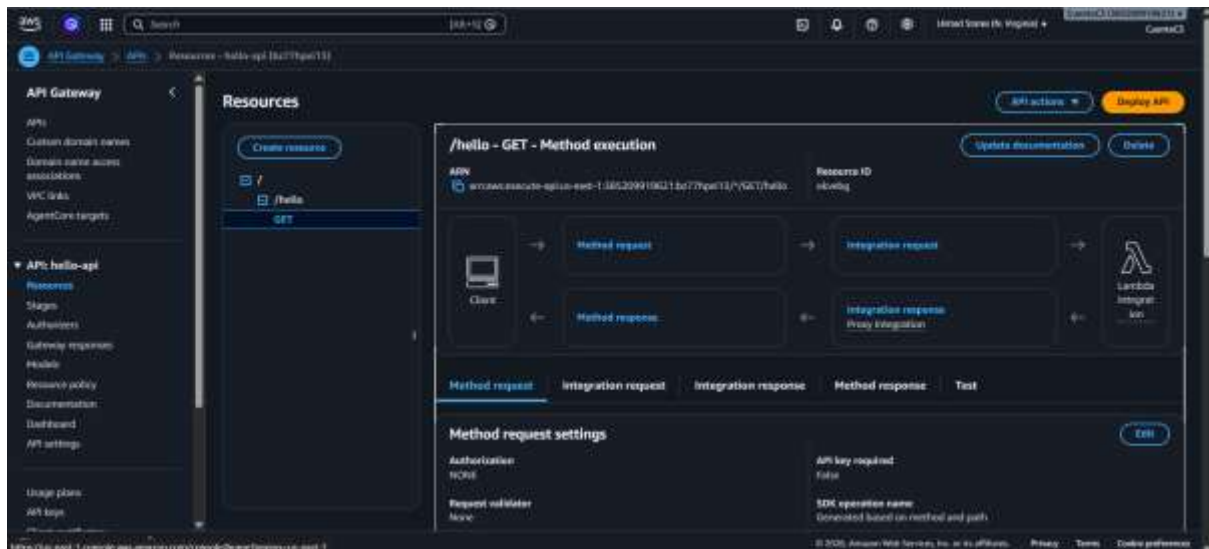


Evidencia 7

Consola AWS mostrando:

- Lambda creada.
- API Gateway creada.
- Integración existente.





Evidencia 8

Tabla de identificación de recursos completada:

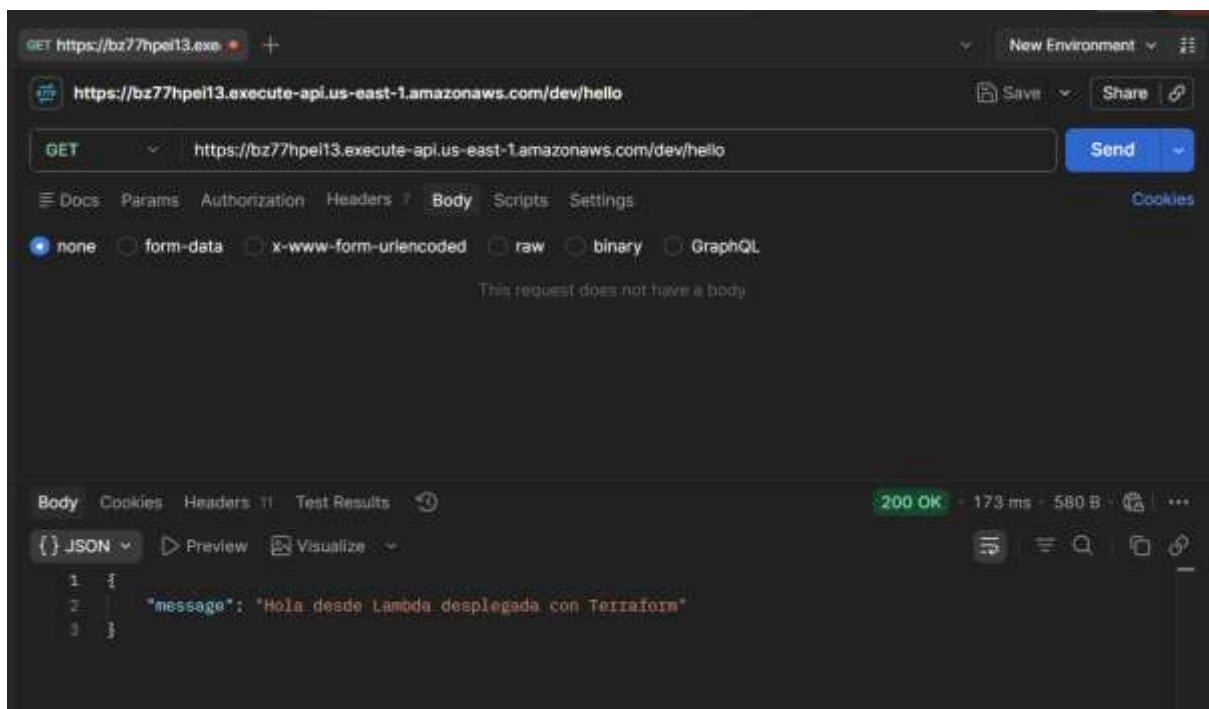
Recurso	Nombre
IAM Role	hello-lambda-role
Lambda	Hello-lambda arn:aws:lambda:us-east-1:385209919621:function:hello-lambda
API Gateway	Hello-api hello-api (bz77hpei13)

Stage	Dev https://bz77hpei13.execute-api.us-east-1.amazonaws.com/dev
Región	Us-east-1

Evidencia 9

Prueba funcional del endpoint desde navegador o Postman mostrando:

- URL invocada.
- Código HTTP 200.
- Respuesta JSON.

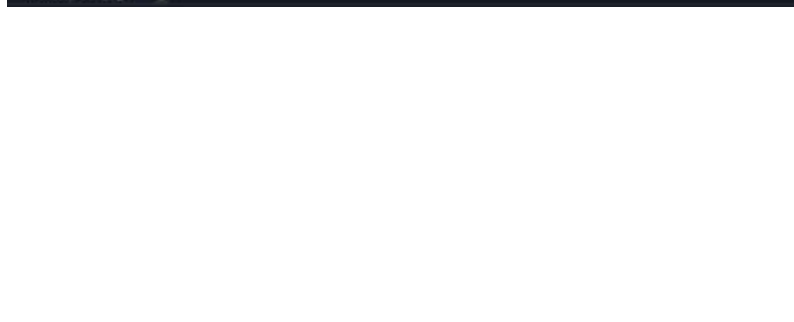
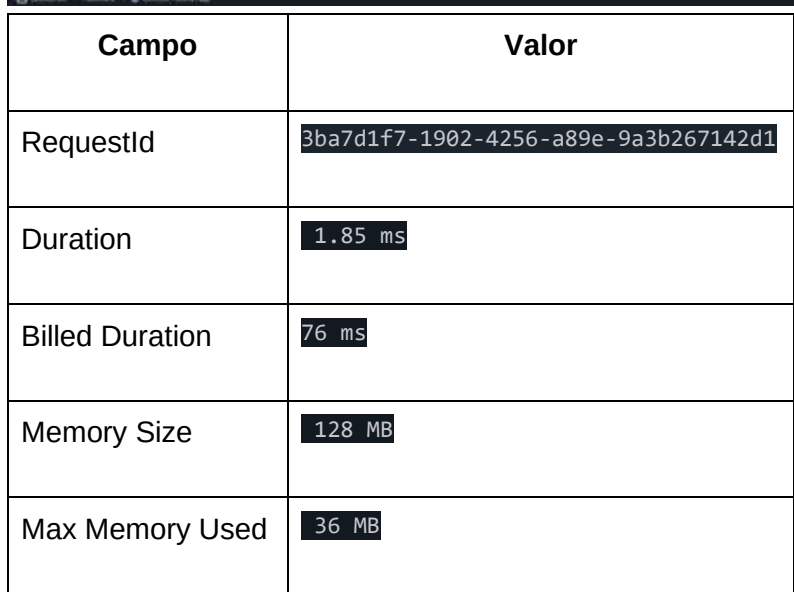


Evidencia 10

CloudWatch Logs mostrando:

- START
- END
- REPORT
- RequestId

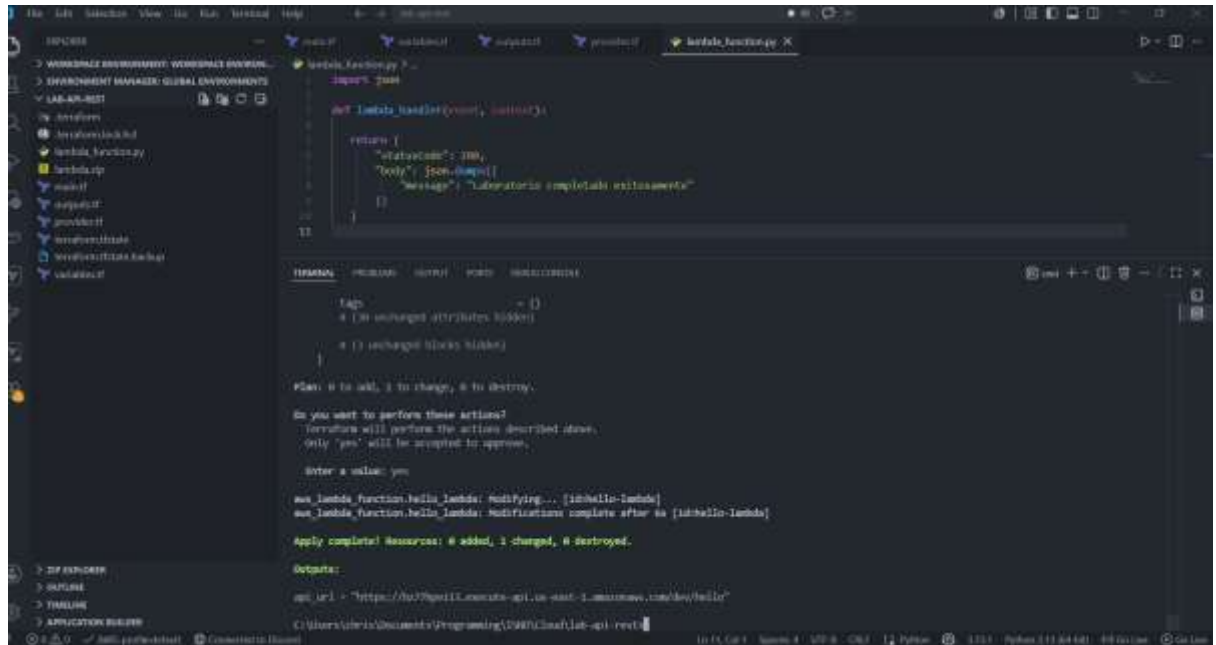
de una ejecución exitosa.



Evidencia 11

Actualización de la función Lambda mostrando:

- Nuevo archivo empaquetado.
- terraform apply ejecutado nuevamente.
- Nuevo resultado desde el endpoint.

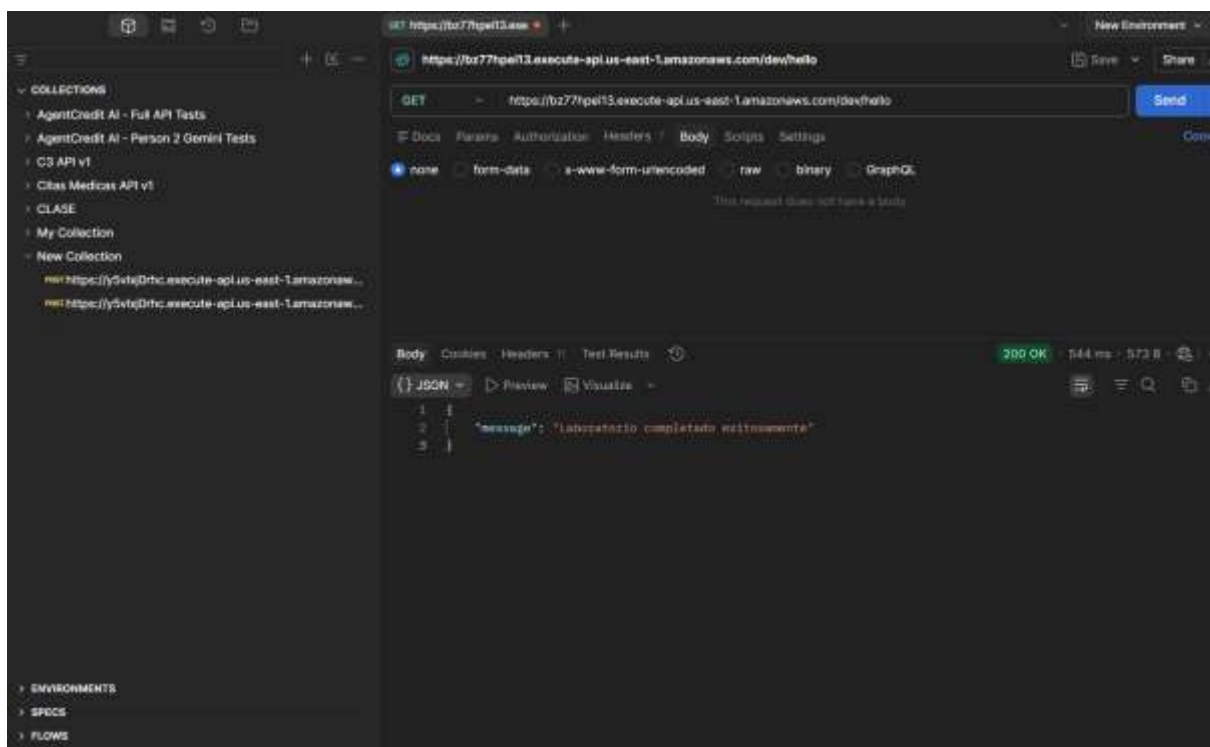


```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,  
        "body": {"foo": "bar"},  
        "message": "Laboratorio completado exitosamente"  
    }  
}
```

terraform apply

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

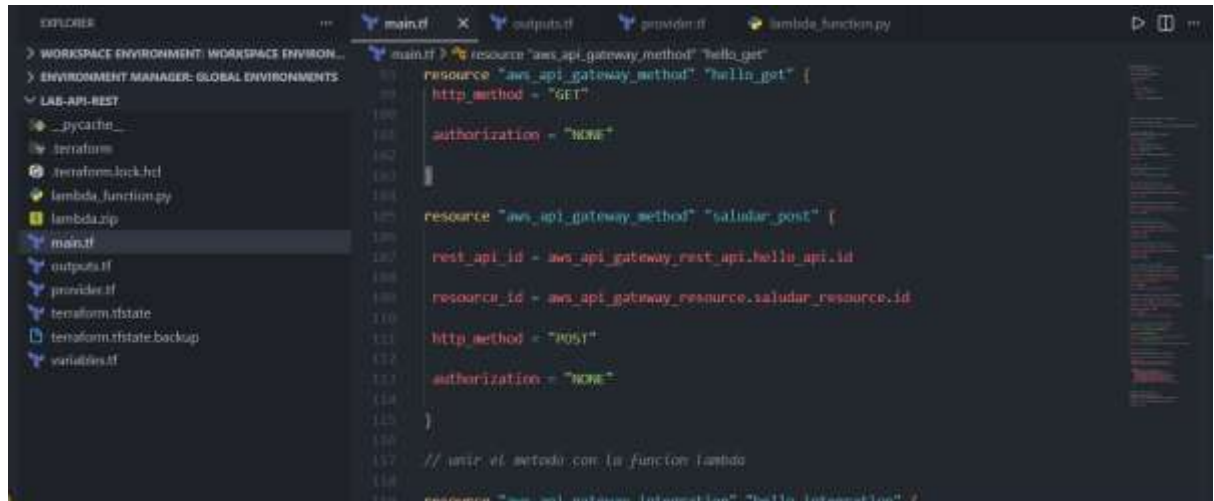
Outputs:
api_url = "https://bz77hpel13.execute-api.us-east-1.amazonaws.com/dev/hello"



Evidencia 12

Actividad de profundización completada mostrando:

- Terraform actualizado.
- Endpoint POST funcionando.
- Validaciones implementadas.



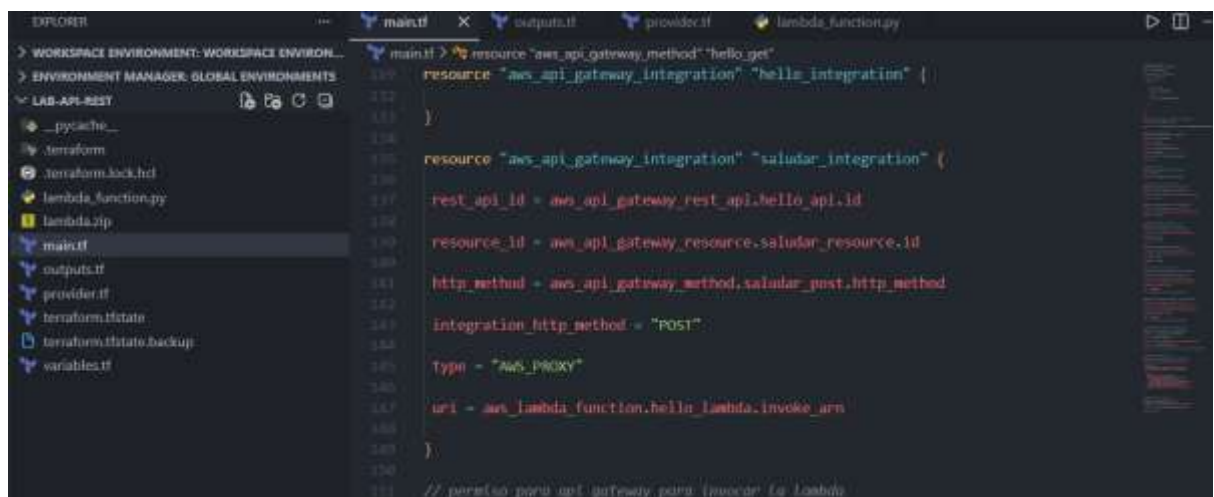
This screenshot shows the VS Code interface with the Terraform configuration file `main.tf` open. The Explorer sidebar on the left shows the project structure for the `LAB-API-REST` workspace, including files like `main.tf`, `outputs.tf`, `provider.tf`, and `lambda_function.py`. The main editor displays the following Terraform code:

```
resource "aws_api_gateway_method" "hello_get" {
  resource "aws_api_gateway_method" "hello_get" {
    http_method = "GET"

    authorization = "NONE"
  }

  resource "aws_api_gateway_method" "saludar_post" {
    rest_api_id = aws_api_gateway_rest_api.hello_api.id
    resource_id = aws_api_gateway_resource.saludar_resource.id
    http_method = "POST"
    authorization = "NONE"
  }

  // unir el metodo con la funcion lambda
  resource "aws_api_gateway_integration" "hello_integration" {
```

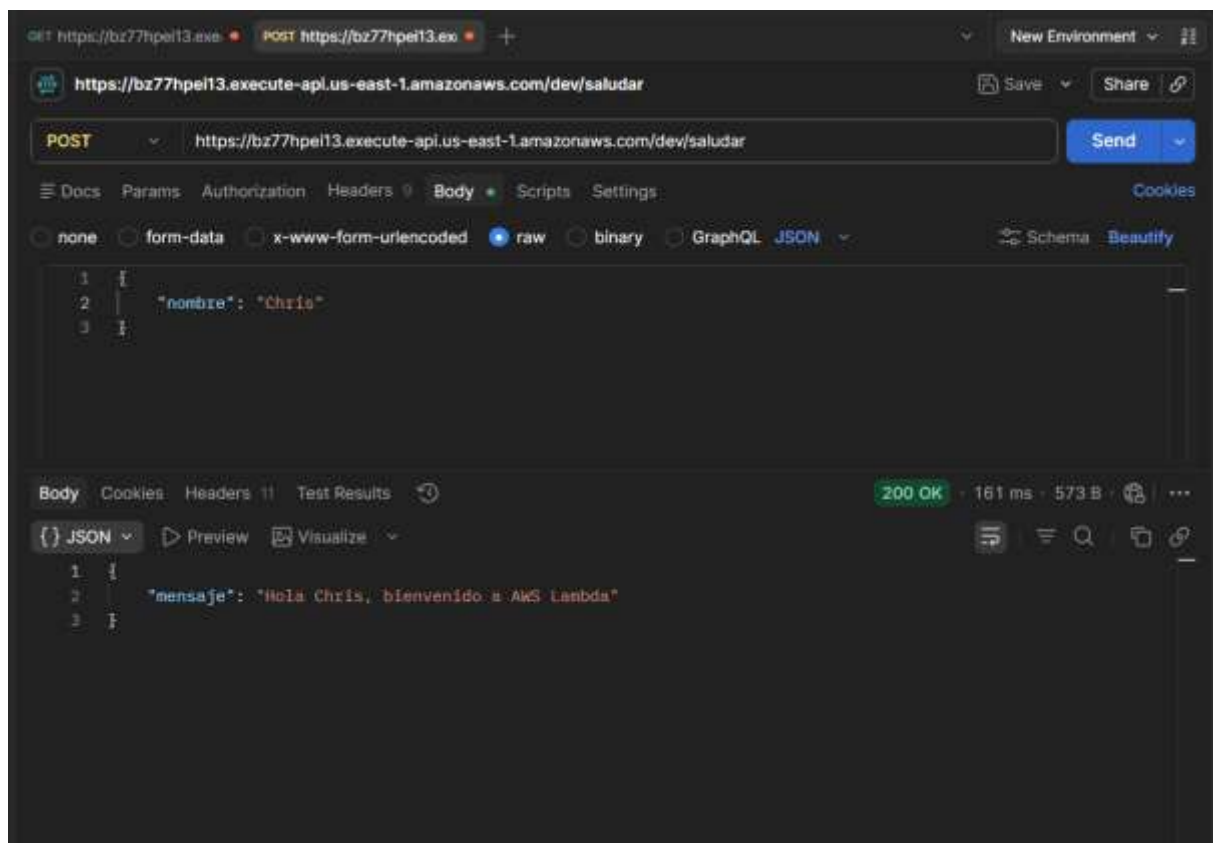
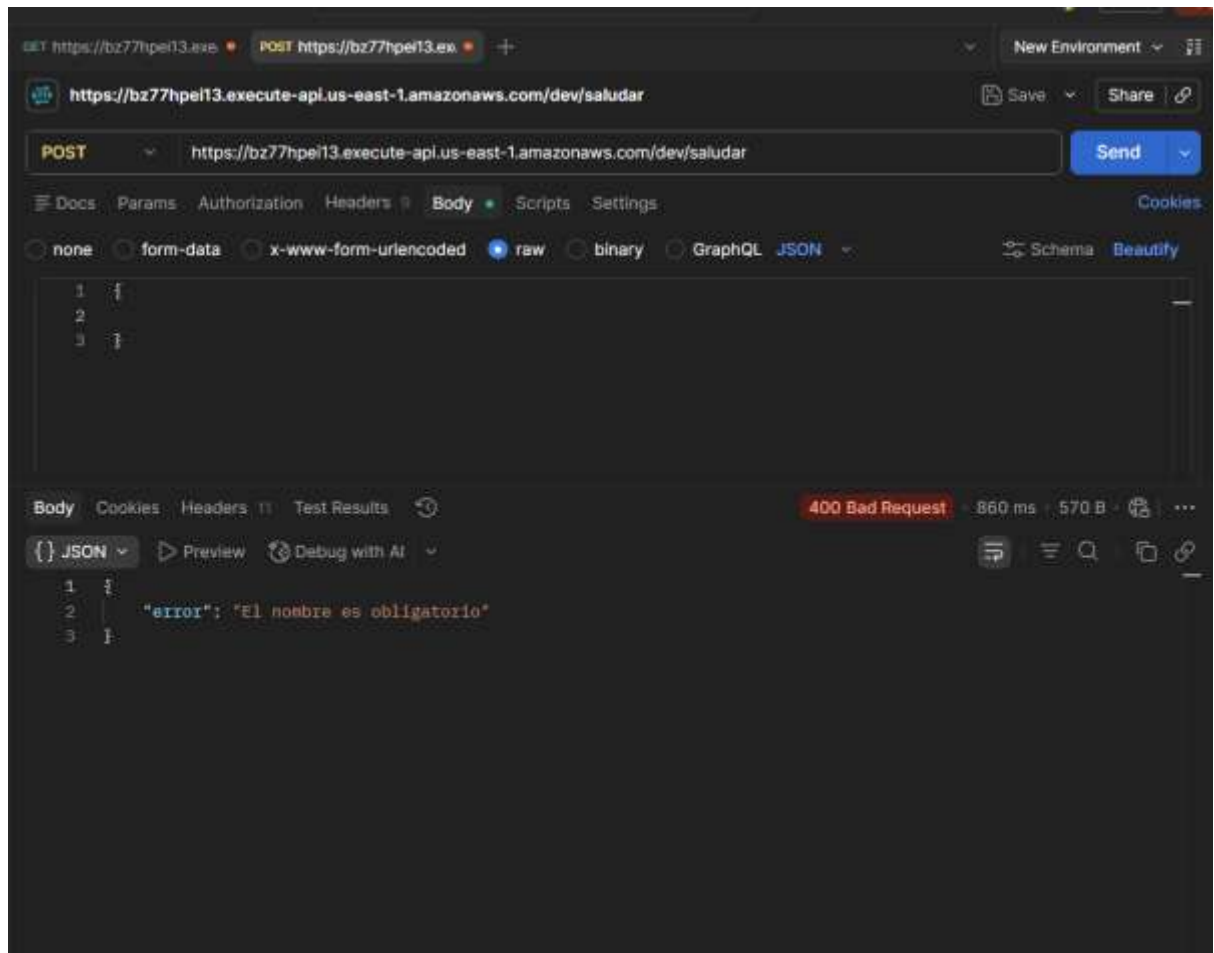


This screenshot shows the continuation of the Terraform configuration in `main.tf`. The code defines two integration resources for the API Gateway:

```
resource "aws_api_gateway_integration" "hello_integration" {
}

resource "aws_api_gateway_integration" "saludar_integration" {
  rest_api_id = aws_api_gateway_rest_api.hello_api.id
  resource_id = aws_api_gateway_resource.saludar_resource.id
  http_method = aws_api_gateway_method.saludar_post.http_method
  integration_http_method = "POST"
  type = "AWS_PROXY"
  uri = aws_lambda_function.hello_lambda.invoke_arn
}

// permiso para api gateway para invocar la lambda
```

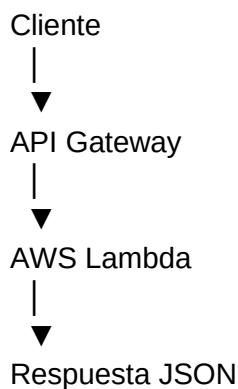



Objetivo

Construir y desplegar una API REST completamente funcional utilizando Terraform para aprovisionar infraestructura AWS.

La solución deberá exponer una función Lambda mediante API Gateway utilizando únicamente infraestructura como código.

Arquitectura



Recursos AWS a Implementar

Durante el laboratorio se crearán los siguientes recursos:

- IAM Role
- IAM Policy Attachment
- AWS Lambda
- API Gateway REST API
- API Gateway Resource
- API Gateway Method
- API Gateway Integration
- API Gateway Deployment
- API Gateway Stage
- CloudWatch Logs (generado automáticamente por Lambda)

Preparación del Proyecto

Crear un directorio de trabajo denominado:

lab-api-rest

Dentro del directorio crear la siguiente estructura:

```
lab-api-rest/  
|  
├── main.tf  
├── variables.tf  
├── outputs.tf  
├── provider.tf  
├── lambda_function.py  
└── lambda.zip
```

Verificar que todos los archivos se encuentren en el mismo directorio.

Guardar Evidencia 1

Creación del Código Lambda

Archivo lambda_function.py

Crear el archivo:

lambda_function.py

Agregar el siguiente código:

```
import json  
  
def lambda_handler(event, context):  
  
    return {  
        "statusCode": 200,  
        "body": json.dumps({  
            "message": "Hola desde Lambda desplegada con Terraform"  
        })  
    }
```

Explicación del Código

La función Lambda está compuesta por:

Elemento	Descripción
----------	-------------

lambda_handler	Punto de entrada de la función
event	Información recibida desde API Gateway
context	Información del entorno de ejecución
statusCode	Código HTTP devuelto
body	Contenido de la respuesta

Empaquetado de la Función

Desde la terminal ejecutar:

Linux / Mac

```
zip lambda.zip lambda_function.py
```

Windows PowerShell

```
Compress-Archive `
-Path lambda_function.py `
-DestinationPath lambda.zip `
-Force
```

Validación del Archivo ZIP

Verificar que exista:

```
lambda.zip
```

La estructura deberá verse similar a:

```
lab-api-rest/
|
|— main.tf
|— variables.tf
|— outputs.tf
|— provider.tf
|— lambda_function.py
|— lambda.zip
```

Si el archivo no existe Terraform no podrá desplegar la función.

Guardar Evidencia 1.

Configuración de Variables

Archivo variables.tf

Crear el archivo: variables.tf

Agregar:

```
variable "aws_region" {  
  
    description = "Región AWS donde se desplegarán los recursos"  
  
    type = string  
  
    default = "us-east-1"  
  
}
```

Configuración del Provider

Archivo provider.tf

Crear el archivo: provider.tf

Agregar:

```
terraform {  
  
    required_version = ">= 1.5"  
  
    required_providers {  
  
        aws = {  
  
            source = "hashicorp/aws"  
  
            version = "~> 6.0"  
  
        }  
  
    }  
  
}
```

```

}

}

provider "aws" {

    region = var.aws_region

}

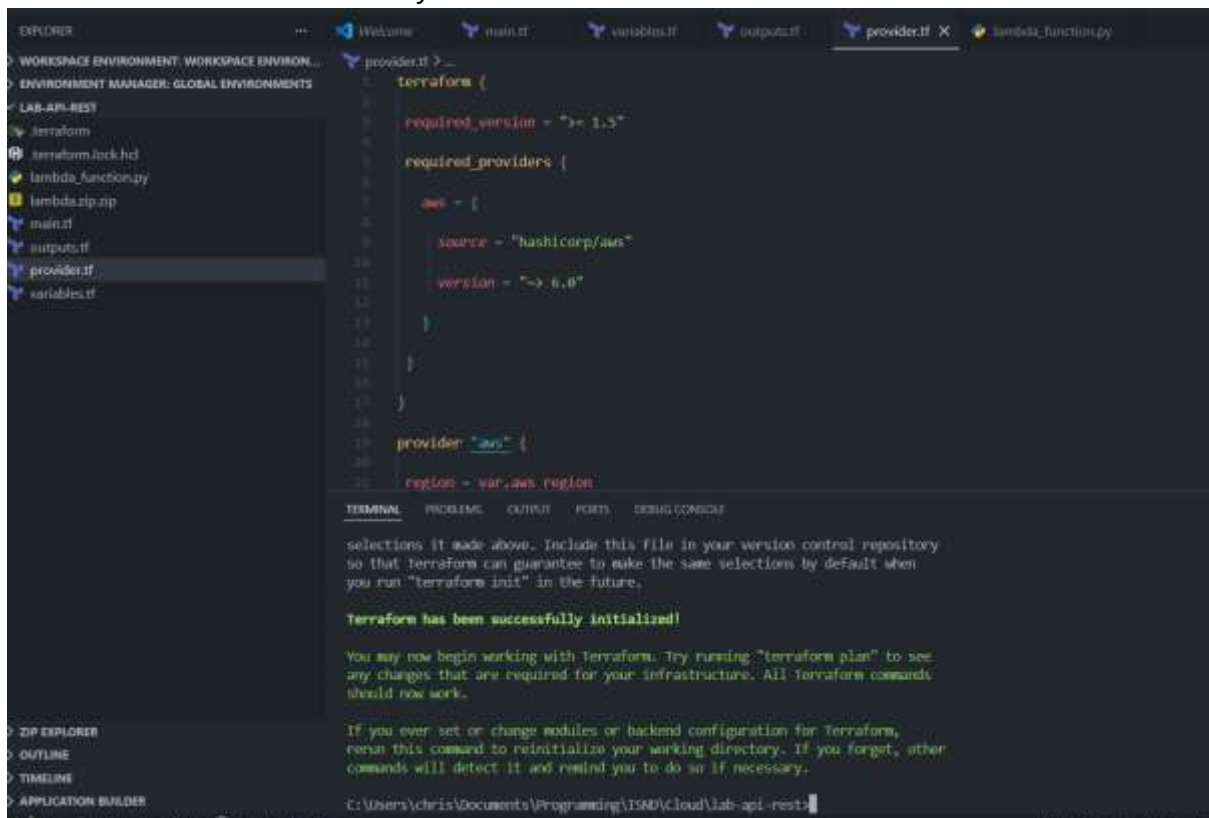
```

Inicialización de Terraform

Ejecutar: terraform init

Resultado esperado:

Terraform has been successfully initialized!



Verificación de Credenciales AWS

Antes de continuar es necesario verificar que Terraform tendrá acceso a AWS.

Ejecutar:

aws sts get-caller-identity

Resultado esperado:

```
{
  "UserId": "...",
  "Account": "...",
  "Arn": "..."
}
```

Análisis del Resultado

Verificar:

- La cuenta AWS es la correcta.
- El usuario posee permisos para crear recursos.
- No existen errores de autenticación.

Si aparece alguno de los siguientes errores:

Unable to locate credentials

o

ExpiredToken

deberán corregirse las credenciales antes de continuar.

Creación del Archivo main.tf

Crear el archivo:

main.tf

Todos los recursos AWS del laboratorio serán agregados en este archivo.

Creación de IAM Role

Agregar al archivo main.tf

```
resource "aws_iam_role" "lambda_role" {
```

```
  name = "hello-lambda-role"
```

```
assume_role_policy = jsonencode({  
  
  Version = "2012-10-17"  
  
  Statement = [  
  
    {  
  
      Effect = "Allow"  
  
      Action = "sts:AssumeRole"  
  
      Principal = {  
  
        Service = "lambda.amazonaws.com"  
  
      }  
  
    }  
  
  ]  
  
})  
}
```

Asociación de Política Administrada

Agregar debajo del recurso anterior:

```
resource "aws_iam_role_policy_attachment" "lambda_logs" {  
  
  role = aws_iam_role.lambda_role.name  
  
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"  
  
}
```

Explicación del IAM Role

El rol creado permitirá:

- Ejecutar funciones Lambda.
- Generar logs en CloudWatch.
- Crear streams de logs.

- Escribir eventos de ejecución.

Sin este rol la función Lambda no podrá ejecutarse.

Validación Parcial

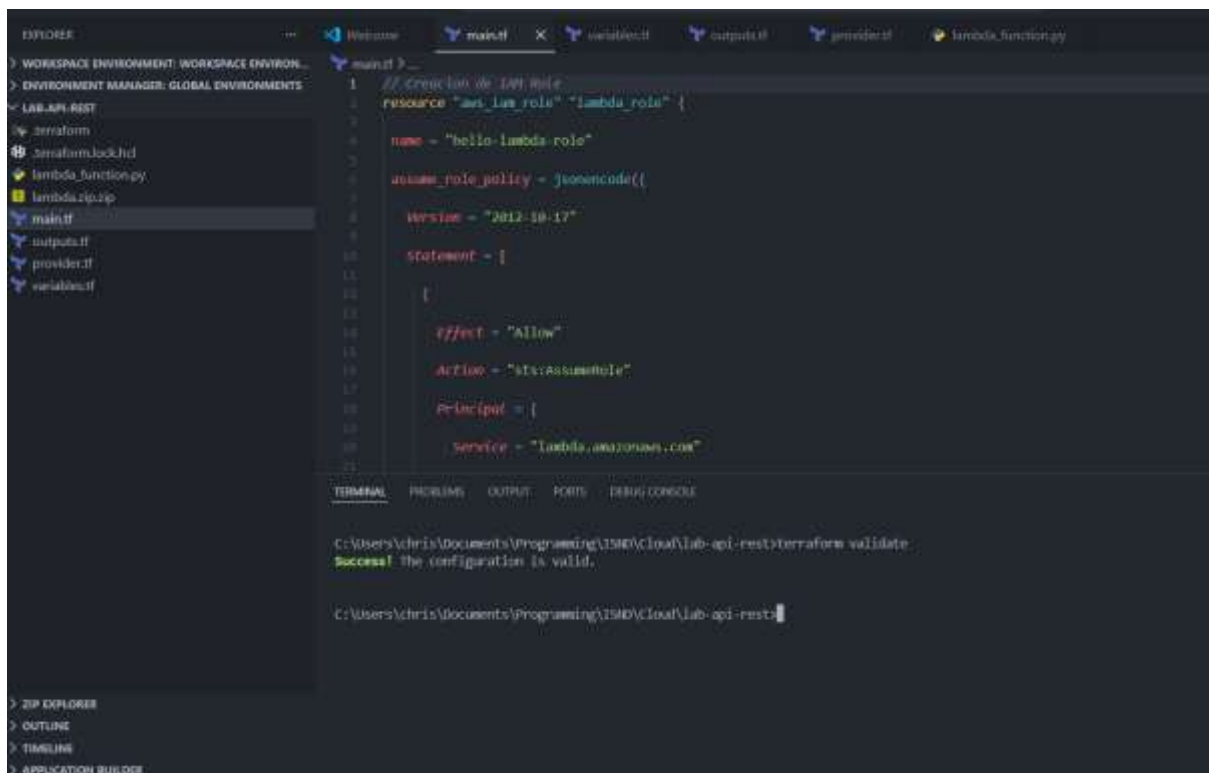
Ejecutar:

terraform validate

Resultado esperado:

Success! The configuration is valid.

Guardar Evidencia 2.



Creación de la Función Lambda

Agregar al archivo `main.tf`:

```
resource "aws_lambda_function" "hello_lambda" {
```

```
    function_name = "hello-lambda"
```

```
    runtime = "python3.12"
```

```
role = aws_iam_role.lambda_role.arn

handler = "lambda_function.lambda_handler"

filename = "lambda.zip"

source_code_hash = filebase64sha256("lambda.zip")

timeout = 10

}
```

Explicación del Recurso Lambda

Propiedad	Descripción
function_name	Nombre de la función
runtime	Lenguaje de ejecución
role	IAM Role utilizado
handler	Punto de entrada
filename	Archivo ZIP
source_code_hash	Detecta cambios
timeout	Tiempo máximo de ejecución

Validación de la Configuración

Ejecutar nuevamente:

```
terraform validate
```

Resultado esperado:

Success! The configuration is valid.

Verificación de Recursos Definidos

Hasta este punto Terraform debería tener definidos:

Recurso	Tipo
lambda_role	IAM Role
lambda_logs	Policy Attachment
hello_lambda	Lambda Function

Verificar que no existan errores de sintaxis antes de continuar.

Creación de la REST API

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_rest_api" "hello_api" {  
  
  name = "hello-api"  
  
  description = "API REST para consumir Lambda"  
  
}
```

Explicación del Recurso

La REST API representa el punto de entrada para los clientes externos.

Terraform creará:

- API Gateway
- Endpoint público
- Recursos internos
- Integraciones

Creación del Recurso /hello

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_resource" "hello_resource" {

  rest_api_id = aws_api_gateway_rest_api.hello_api.id

  parent_id = aws_api_gateway_rest_api.hello_api.root_resource_id

  path_part = "hello"

}
```

Explicación del Recurso

El atributo:

```
path_part = "hello"
```

creará la ruta:

```
/hello
```

La URL final tendrá una estructura similar a:

```
https://xxxxxxxx.execute-api.us-east-1.amazonaws.com/dev/hello
```

Creación del Método GET

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_method" "hello_get" {

  rest_api_id = aws_api_gateway_rest_api.hello_api.id

  resource_id = aws_api_gateway_resource.hello_resource.id

  http_method = "GET"

  authorization = "NONE"

}
```

Explicación del Método

Se está configurando:

Parámetro	Valor
Método HTTP	GET
Autenticación	NONE

Esto permitirá acceder a la API sin autenticación.

Integración de API Gateway con Lambda

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_integration" "hello_integration" {  
  
  rest_api_id = aws_api_gateway_rest_api.hello_api.id  
  
  resource_id = aws_api_gateway_resource.hello_resource.id  
  
  http_method = aws_api_gateway_method.hello_get.http_method  
  
  integration_http_method = "POST"  
  
  type = "AWS_PROXY"  
  
  uri = aws_lambda_function.hello_lambda.invoke_arn  
  
}
```

Explicación de la Integración

Parámetro	Función
AWS_PROXY	Envía la solicitud completa a Lambda
POST	Método interno usado por API Gateway
invoke_arn	Dirección de invocación de Lambda

Cuando un cliente invoque:

GET /hello

API Gateway enviará automáticamente la solicitud a Lambda.

Permisos para API Gateway

Agregar al archivo **main.tf**:

```
resource "aws_lambda_permission" "allow_gateway" {  
  
  statement_id = "AllowExecutionFromAPIGateway"  
  
  action = "lambda:InvokeFunction"  
  
  function_name = aws_lambda_function.hello_lambda.function_name  
  
  principal = "apigateway.amazonaws.com"  
  
  source_arn = "${aws_api_gateway_rest_api.hello_api.execution_arn}/*/*"  
  
}
```

Explicación del Permiso

Este recurso autoriza a API Gateway para ejecutar la función Lambda.

Sin este permiso se produciría un error similar a:

Execution failed due to configuration error

o

Invalid permissions on Lambda function

Validación de Configuración

Ejecutar:

```
terraform validate
```

Resultado esperado:

Success! The configuration is valid.

Despliegue de la API

Deployment

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_deployment" "deployment" {

  rest_api_id = aws_api_gateway_rest_api.hello_api.id

  depends_on = [
    aws_api_gateway_integration.hello_integration
  ]

  triggers = {
    redeployment = sha1(jsonencode([
      aws_api_gateway_resource.hello_resource.id,
      aws_api_gateway_method.hello_get.id,
      aws_api_gateway_integration.hello_integration.id
    ]))
  }
}
```

Explicación del Deployment

El deployment publica la API.

Sin este recurso:

- La API existiría.
- El endpoint no estaría disponible.

Creación del Stage

Agregar al archivo **main.tf**:

```
resource "aws_api_gateway_stage" "dev" {

  deployment_id = aws_api_gateway_deployment.deployment.id

  rest_api_id = aws_api_gateway_rest_api.hello_api.id
}
```

```
stage_name = "dev"

}
```

Explicación del Stage

El Stage representa una versión publicada de la API.

En este laboratorio se utilizará:

dev

La URL contendrá:

/dev

Creación de Outputs

Archivo outputs.tf

Agregar:

```
output "api_url" {

  description = "URL pública de la API"

  value = "https://${aws_api_gateway_rest_api.hello_api.id}.execute-
api.${var.aws_region}.amazonaws.com/dev/hello"

}
```

Validación Final de Terraform

Ejecutar:

```
terraform validate
```

Resultado esperado:

Success! The configuration is valid.

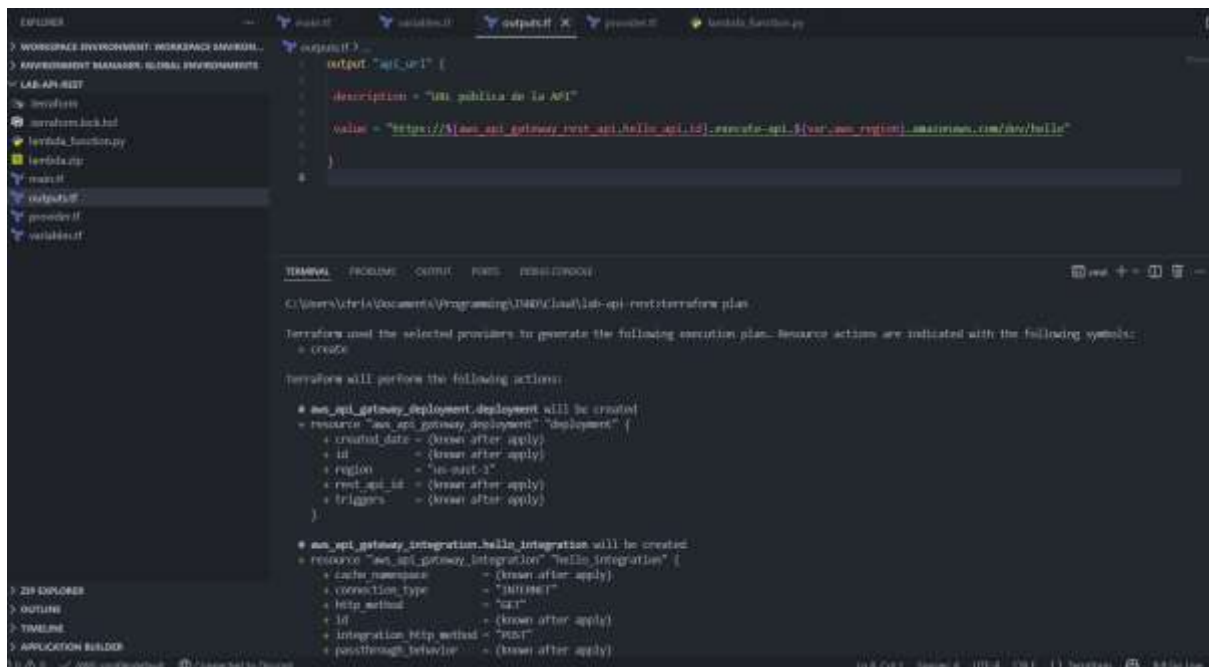
Guardar Evidencia 2.

Generación del Plan

Ejecutar:

terraform plan

Guardar Evidencia 3.



```
output "api_url" {
  description = "URI pública de la API"
  value       = "https://{{aws_api_gateway_rest_api.hello_api.id}}.execute-api-{{var.aws_region}}.amazonaws.com/hello"
}
```

Terraform will perform the following actions:

- `aws_api_gateway_deployment.deployment` will be created
- resource "aws_api_gateway_deployment" "deployment" {
 - `created_date` = (known after apply)
 - `id` = (known after apply)
 - `region` = "us-east-1"
 - `rest_api_id` = (known after apply)
 - `triggers` = (known after apply)
- `aws_api_gateway_integration.hello_integration` will be created
- resource "aws_api_gateway_integration" "hello_integration" {
 - `cache_namespace` = (known after apply)
 - `connection_type` = "INTERNET"
 - `http_method` = "GET"
 - `id` = (known after apply)
 - `integration_http_method` = "POST"
 - `passthrough_behavior` = (known after apply)

Análisis del Plan

Analizar la salida generada por Terraform.

Identificar:

- Cantidad total de recursos.
- Nombre del IAM Role.
- Nombre de la Lambda.
- Nombre de la API.
- Nombre del Stage.

Completar la siguiente tabla:

Elemento	Valor
IAM Role	"hello-lambda-role"

Lambda	"hello-lambda"
API	"hello-api"
Stage	"dev"
Recursos a crear	

Guardar Evidencia 4.

The screenshot shows a VS Code editor with the following files in the Explorer: `main.tf`, `variables.tf`, `outputs.tf`, `provider.tf`, and `lambda_function.py`. The `outputs.tf` file is open, showing the following configuration:

```

output "api_url" {
  description = "URL pública de la API"
  value = "https://${aws_api_gateway_rest_api.hello_api.id}.execute-api.${var.aws_region}"
}

```

The terminal output shows the Terraform plan for the `aws_lambda_permission.allow_gateway` resource:

```

# aws_lambda_permission.allow_gateway will be created
+ resource "aws_lambda_permission" "allow_gateway" {
+   action          = "lambda:InvokeFunction"
+   function_name   = "hello-lambda"
+   id              = (known after apply)
+   principal       = "apigateway.amazonaws.com"
+   region         = "us-east-1"
+   source_arn      = (known after apply)
+   statement_id    = "AllowExecutionFromAPIGateway"
+   statement_id_prefix = (known after apply)
}

Plan: 10 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ api_url = (known after apply)

```

A note at the bottom of the terminal states: "Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly". The file path at the bottom is `C:\Users\chris\Documents\Programming\IISD\Cloud\lab-api-rest`.

Aplicación de la Infraestructura

Ejecutar:

terraform apply

Terraform mostrará un resumen similar a:

Plan: X to add, 0 to change, 0 to destroy.

Responder:

yes

Esperar la finalización del proceso.

Resultado Esperado

Terraform deberá mostrar mensajes similares a:

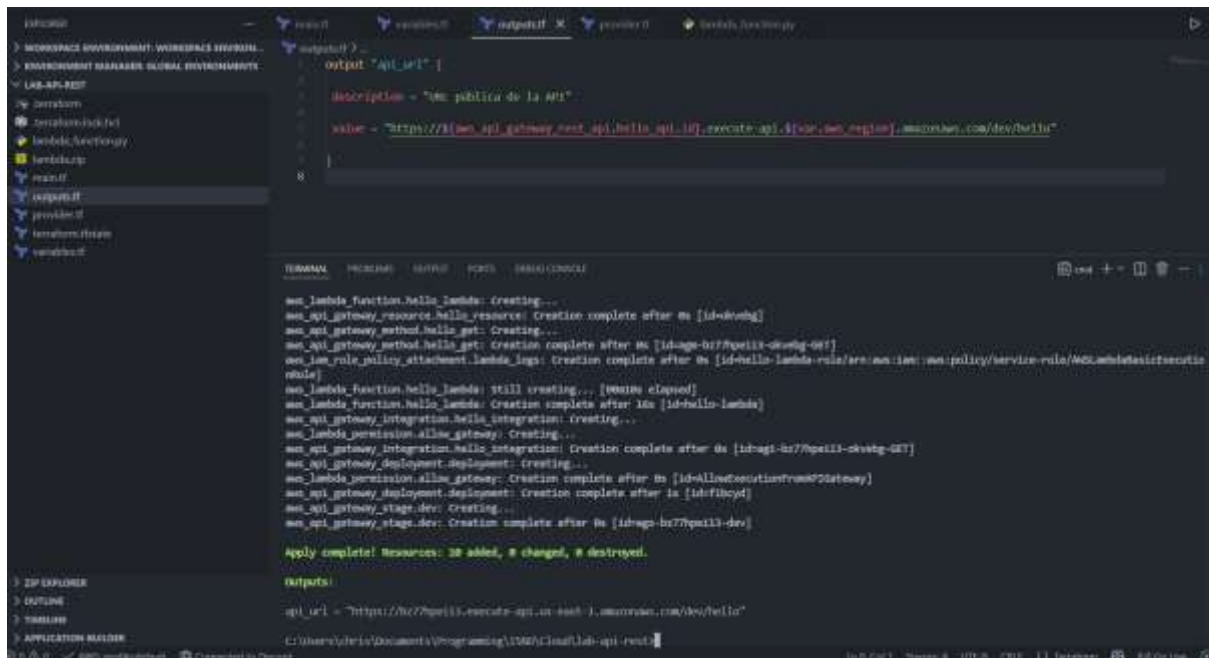
Apply complete!

y posteriormente:

Outputs:

api_url = https://xxxxxxx.execute-api.us-east-1.amazonaws.com/dev/hello

Guardar Evidencia 5.



```
terraform apply

aws_lambda_function.hello_lambda: creating...
aws_api_gateway_resource.hello_resource: Creation complete after 0s [id=dkwbg]
aws_api_gateway_method.hello_get: Creating...
aws_api_gateway_method.hello_get: Creation complete after 0s [id=ag-bc77qpsll-0kwhg-0ET]
aws_lambda_function.hello_lambda: still creating... [waiting elapsed]
aws_lambda_function.hello_lambda: Creation complete after 10s [id=hello-lambda]
aws_api_gateway_integration.hello_integration: creating...
aws_lambda_permission.allow_gateway: Creating...
aws_api_gateway_integration.hello_integration: Creation complete after 0s [id=agi-bc77qpsll-0kwhg-0ET]
aws_api_gateway_deployment.deployment: creating...
aws_lambda_permission.allow_gateway: Creation complete after 0s [id=allowexecutionfrom720gateway]
aws_api_gateway_deployment.deployment: Creation complete after 1s [id=fhcyd]
aws_api_gateway_stage.dev: Creating...
aws_api_gateway_stage.dev: Creation complete after 0s [id=sg-bc77qpsll-dev]

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

api_url = "https://bc77qpsll.execute-api.us-east-1.amazonaws.com/dev/hello"

C:\Users\alexis\Documents\Programming\TMD\Cloud\lab-api-rest>
```

Verificación Inicial del Endpoint

Copiar la URL mostrada por Terraform.

Ejemplo:

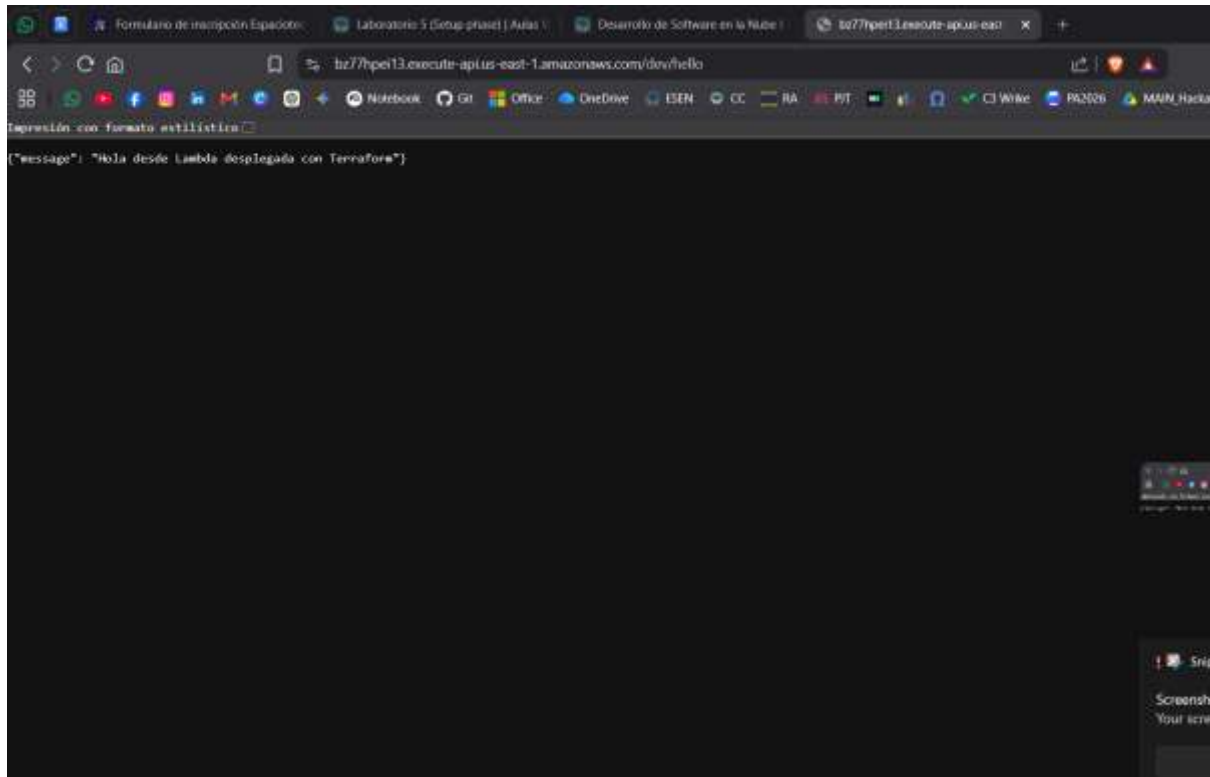
https://xxxxxxx.execute-api.us-east-1.amazonaws.com/dev/hello

Abrir la URL en el navegador.

Resultado esperado:

```
{  
  "message": "Hola desde Lambda desplegada con Terraform"  
}
```

Si el endpoint responde correctamente, continuar con la siguiente sección.



Exploración de Recursos AWS

Tiempo estimado: 10 minutos

Objetivo

Verificar que Terraform haya creado correctamente todos los recursos en AWS.

Ingresar a la consola AWS únicamente para actividades de validación.

No se permite crear ni modificar recursos desde la consola.

Verificación del IAM Role

Ingresar al servicio:

IAM

Seleccionar:

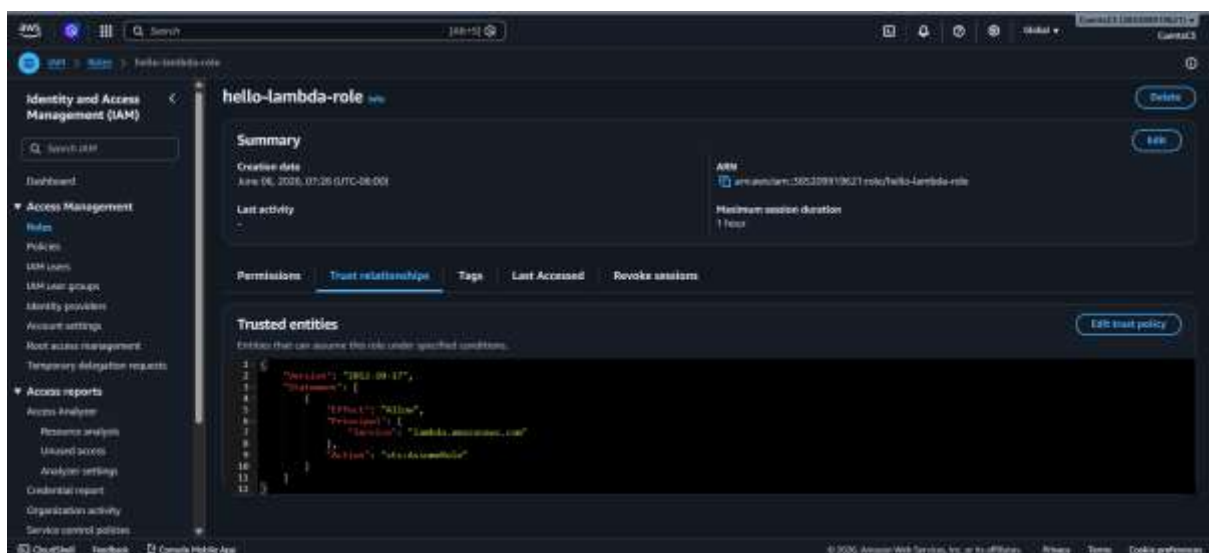
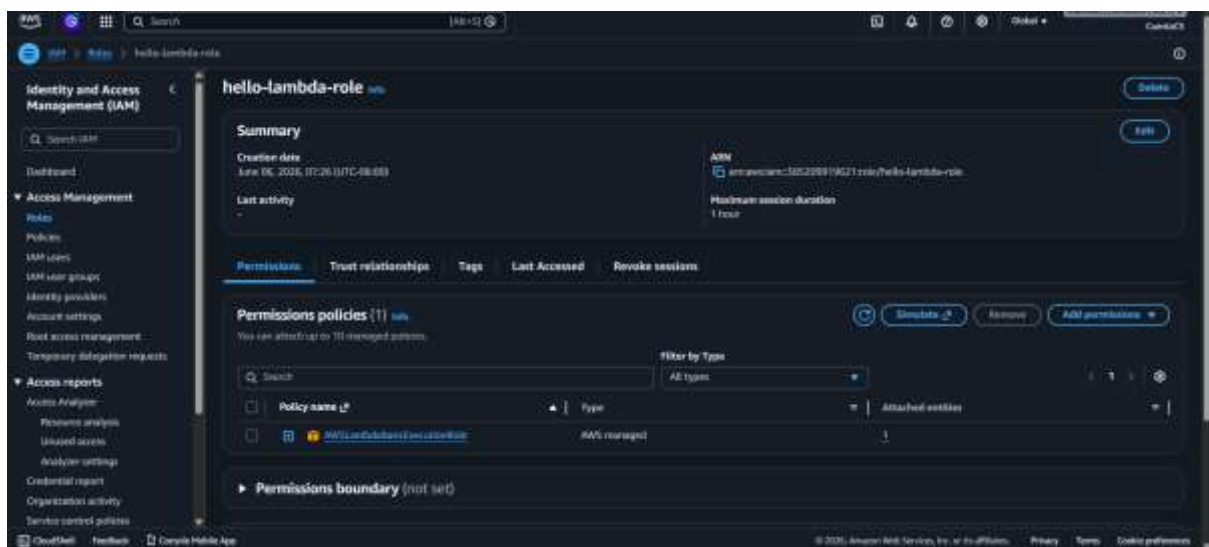
Roles

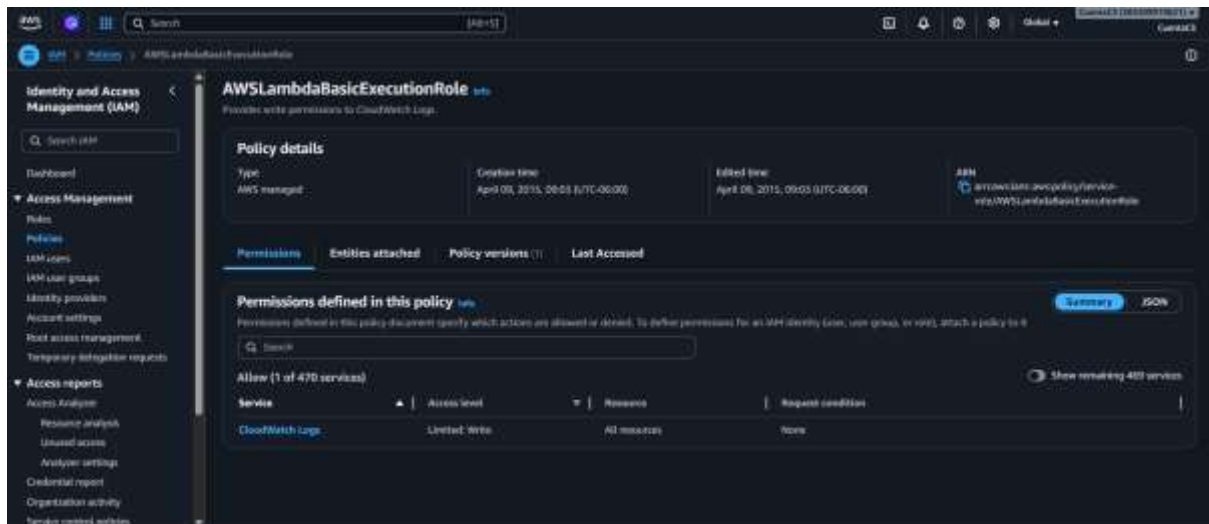
Ubicar el rol:

hello-lambda-role

Verificar:

- Nombre del rol.
- Relación de confianza (Trust Relationship).
- Política adjunta AWSLambdaBasicExecutionRole.





Verificación de Lambda

Ingresar al servicio:

Lambda

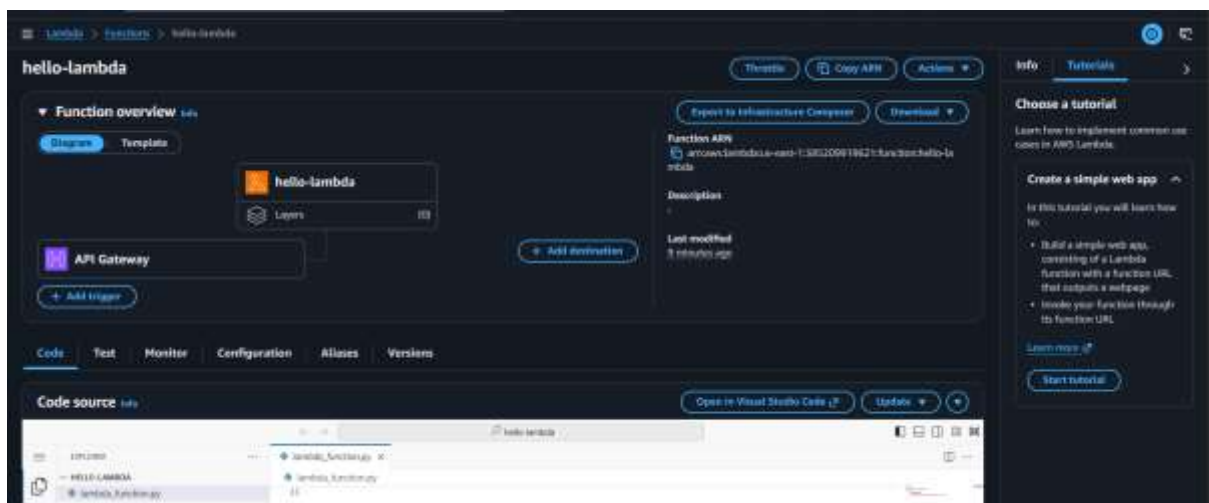
Ubicar la función:

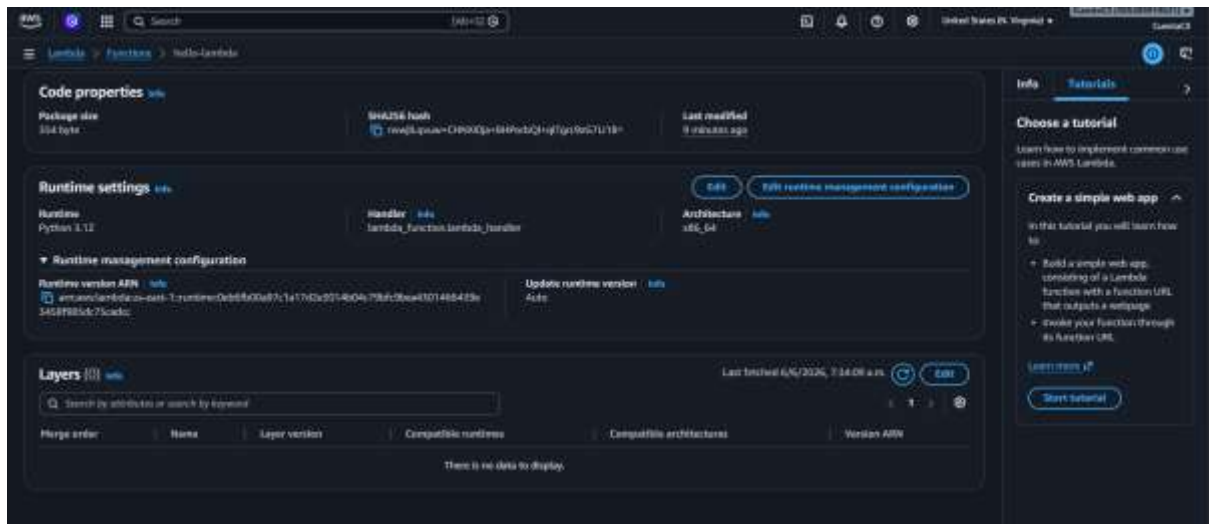
hello-lambda

Verificar:

- Runtime Python 3.12.
- Handler.
- Timeout.
- IAM Role asociado.

Guardar evidencia.





Verificación de API Gateway

Ingresar al servicio:

API Gateway

Ubicar:

hello-api

Verificar:

- Recurso /hello.
- Método GET.
- Integración Lambda.
- Stage dev.

Guardar Evidencia 7.

Identificación de Recursos

Completar la siguiente tabla:

Recurso	Nombre
IAM Role	hello-lambda-role
Lambda	Hello-lambda

	arn:aws:lambda:us-east-1:385209919621:function:hello-lambda
API Gateway	Hello-api hello-api (bz77hpei13)
Stage	Dev https://bz77hpei13.execute-api.us-east-1.amazonaws.com/dev
Región	Us-east-1

Guardar Evidencia 8.

Invocación Directa de Lambda

Crear Evento de Prueba

Ingresar a:

Lambda

Seleccionar:

hello-lambda

Presionar:

Test

Crear un evento con el siguiente contenido:

```
{}
```

Guardar el evento con el nombre:

evento-prueba

Ejecutar la Prueba

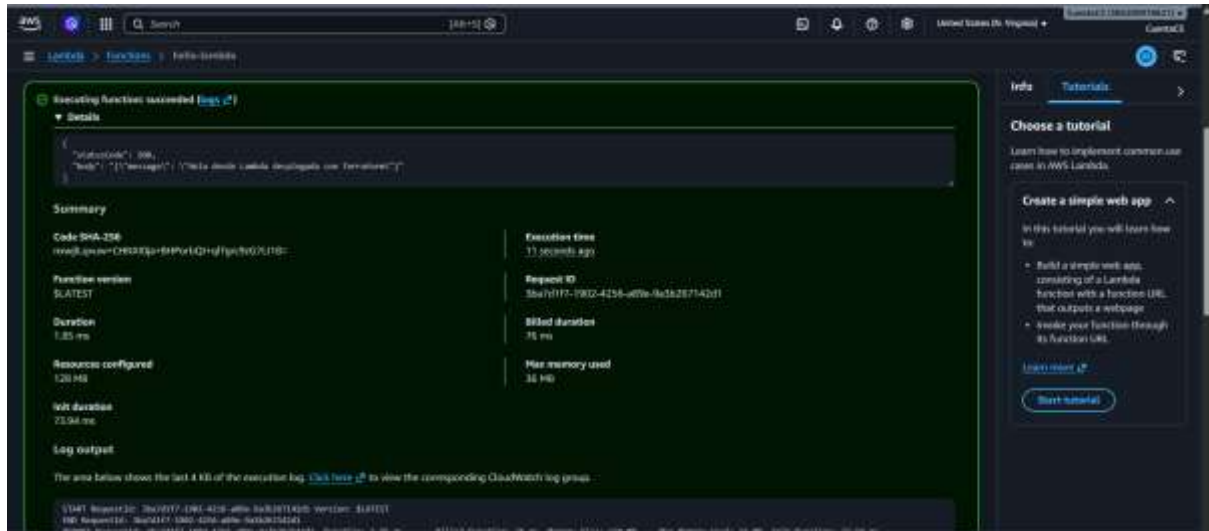
Presionar:

Test

Verificar el resultado.

Respuesta esperada:

```
{
  "statusCode": 200,
  "body": "{\"message\":\"Hola desde Lambda desplegada con Terraform\"}"
}
```



Analizar Métricas de Ejecución

Identificar:

- Duration
- Billed Duration
- RequestId

Guardar Evidencia 6.

Exploración de CloudWatch

Acceso a CloudWatch

Ingresar al servicio:

CloudWatch

Seleccionar:

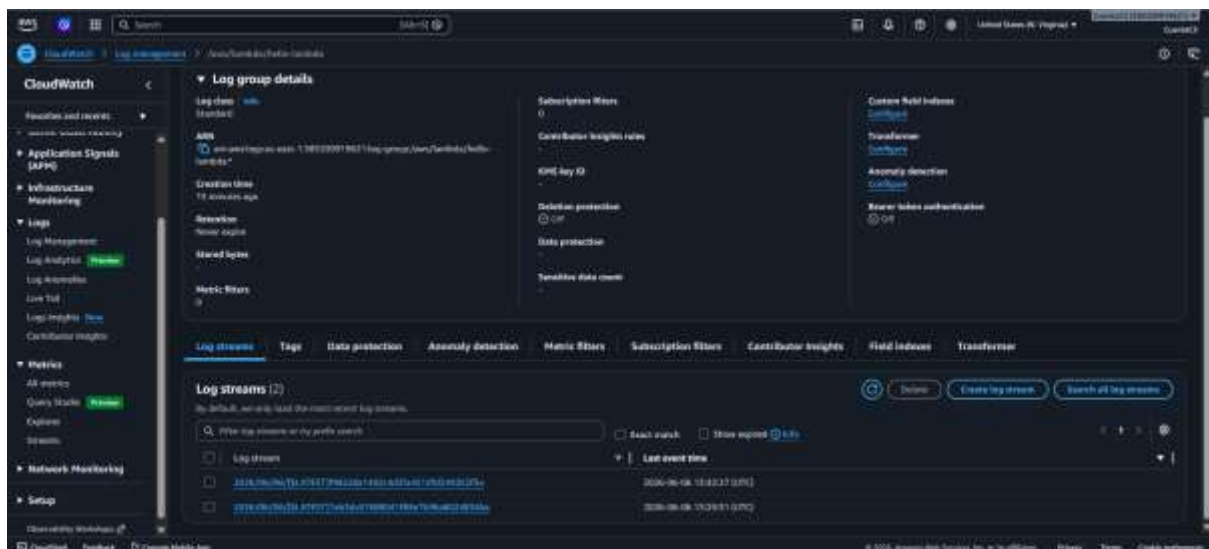
Log Groups

```
/ aws / lambda / hello-lambda
```

Revisar Logs

Identificar las siguientes entradas:

REPORT RequestId

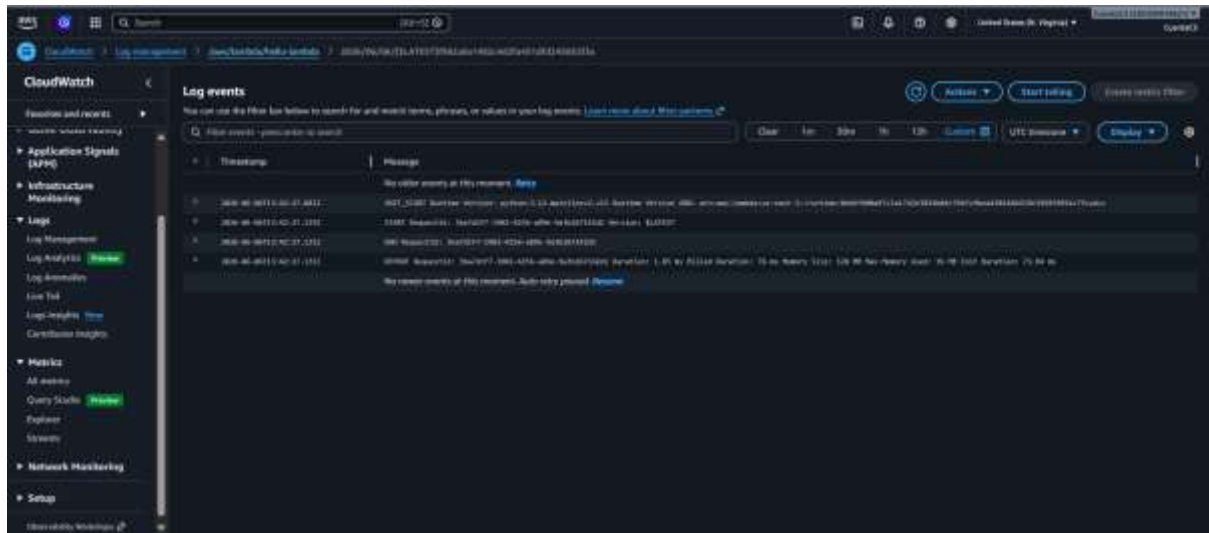


Análisis del Log

Campo	Valor
RequestId	3ba7d1f7-1902-4256-a89e-9a3b267142d1
Duration	1.85 ms
Billed Duration	76 ms

Memory Size	128 MB
Max Memory Used	36 MB

Guardar Evidencia 10.



Pruebas Funcionales

Obtención del Endpoint

Copiar la URL generada por Terraform.

Ejemplo:

<https://xxxxxxxx.execute-api.us-east-1.amazonaws.com/dev/hello>

Prueba desde Navegador

Abrir la URL.

Verificar respuesta:

```
{
  "message": "Hola desde Lambda desplegada con Terraform"
}
```

Prueba desde Postman

Crear una petición:

GET

Hacia:

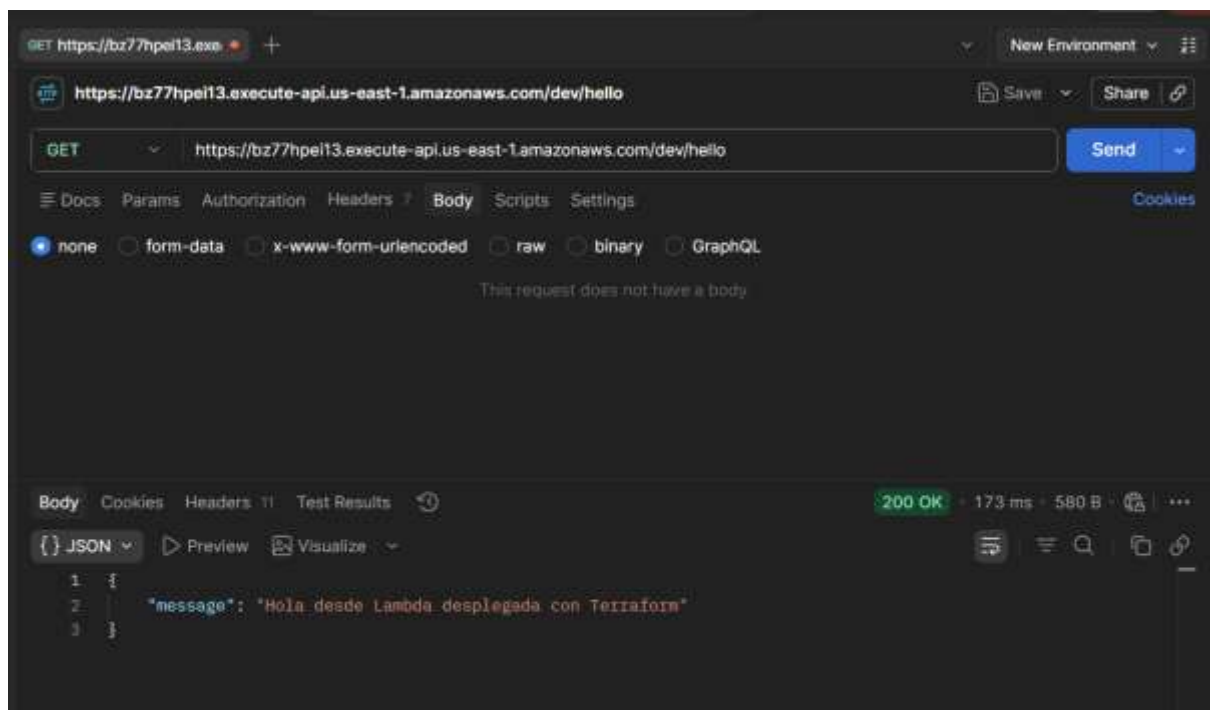
<https://xxxxxxx.execute-api.us-east-1.amazonaws.com/dev/hello>

Enviar la solicitud.

Verificar:

- Código HTTP 200.
- Respuesta JSON.

Guardar Evidencia 9.



Actualización de la Función Lambda

Modificar lambda_function.py

Reemplazar el contenido por:

```
import json
```

```
def lambda_handler(event, context):

    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "Laboratorio completado exitosamente"
        })
    }
```

Regenerar el Archivo ZIP

Ejecutar:

Linux / Mac

```
zip lambda.zip lambda_function.py
```

Windows PowerShell

```
Compress-Archive `
-Path lambda_function.py `
-DestinationPath lambda.zip `
-Force
```

Aplicar Cambios

Ejecutar:

```
terraform plan
```

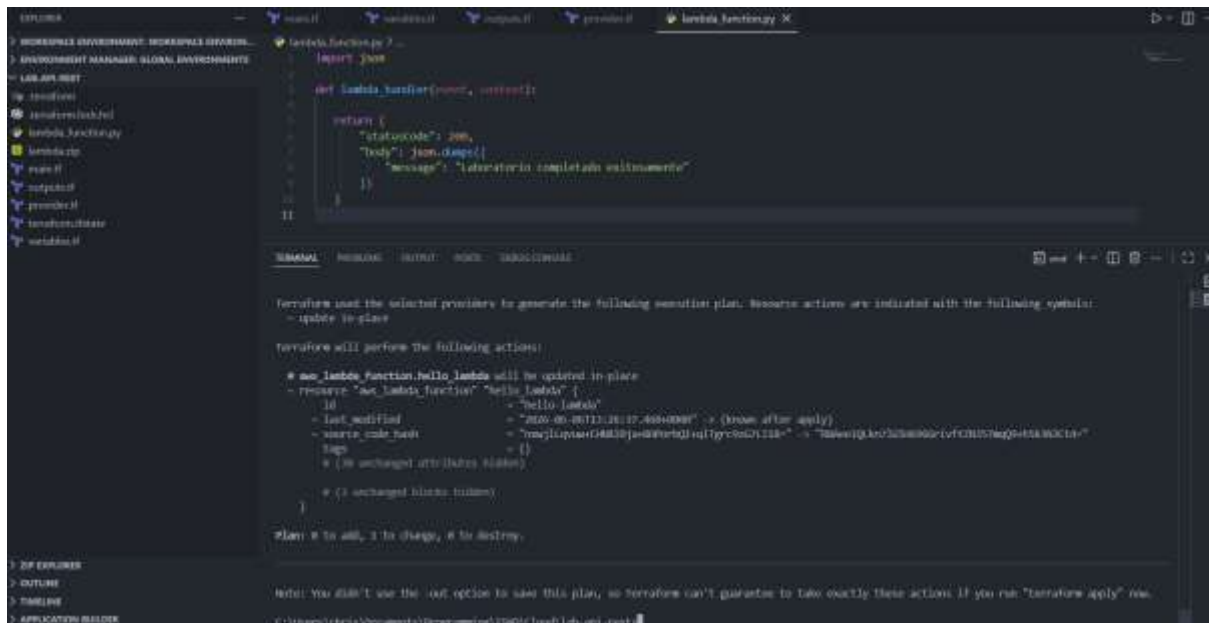
Observar que Terraform detecta cambios en Lambda.

Posteriormente ejecutar:

```
terraform apply
```

Responder:

Yes



Validar Cambios

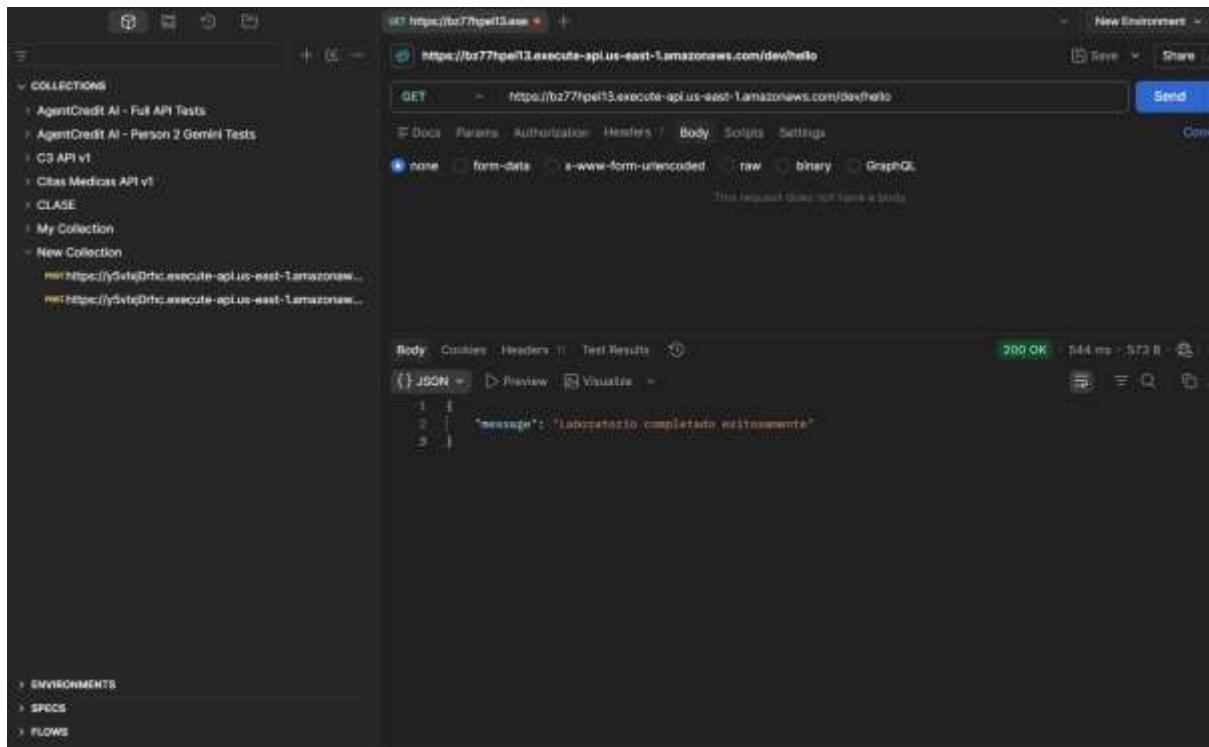
Invocar nuevamente:

GET /hello

Respuesta esperada:

```
{
  "message": "Laboratorio completado exitosamente"
}
```

Guardar Evidencia 11.



Actividad de Profundización

Nuevo Endpoint

Implementar:

POST /saludar

Entrada Esperada

```
{  
  "nombre": "Juan"  
}
```

Respuesta Esperada

```
{  
  "mensaje": "Hola Juan, bienvenido a AWS Lambda"  
}
```

Requisitos Técnicos

La solución deberá:

- Utilizar Terraform para todos los cambios.
- Crear los recursos necesarios en API Gateway.
- Modificar Lambda para procesar información dinámica.
- Mantener la integración AWS_PROXY.
- No utilizar la consola AWS para crear recursos.

Validaciones Requeridas

Implementar manejo de errores para:

- Nombre vacío.
- Nombre nulo.
- JSON inválido.

Respuesta esperada:

```
{  
  "error": "El nombre es obligatorio"  
}
```

Código HTTP esperado:

400 Bad Request

Evidencias de la Actividad de Profundización

Presentar:

- Código Terraform actualizado.
- Cambios realizados en Lambda.
- Resultado de terraform apply.
- Endpoint POST funcionando.
- Ejemplo de respuesta exitosa.
- Ejemplo de respuesta con error.

Guardar Evidencia 12.